Data Handling

Input Schema

The pipeline ingests three CSV files exported from the academy's student information system. Each file is read with explicit dtype overrides to prevent pandas from inferring types that would corrupt IDs and phone numbers. All column names are defined as constants in src/config.py and never hardcoded in logic modules.

- student_metadata.csv - Five columns: student_id (str), student_name (str), campus_id (str), parent_phone (str), facilitator_email (str). dtype overrides applied to student_id and parent_phone to prevent leading-zero loss and float64 coercion.

- daily_metrics.csv - Four columns: student_id (str), metric_date (str), session_attended_min (numeric), practice_questions (numeric). Metric date is aggregated by student; session and practice columns are summed and averaged.

- facilitator_notes.csv - Three columns: student_id (str), note_date (str), note_text (str). The most recent note date per student is extracted and used to compute days_since_last_note.

- Explicit dtype loading pattern: pd.read_csv(path, dtype={"student_id": "str", "parent_phone": "str"}) applied to every read_csv call - never let pandas infer dtypes for ID or phone columns.

Data Cleaning Pipeline

After loading each CSV, ingestion.py applies a four-step cleaning pipeline before merging. The goal is to produce a single clean DataFrame where every student appears exactly once, every numeric column has a finite value, and no sentinel strings like "N/A" or "none" appear in numeric positions.

- Step 1 - Type coercion: all numeric metric columns (session_attended_min, practice_questions) are loaded as str dtype, then converted to numeric via pd.to_numeric(errors='coerce'). This safely converts type-mismatch strings (e.g., "none", "abc") to NaN without crashing the pipeline.

- Step 2 - Deduplication: the merged DataFrame is deduplicated on student_id by keeping the first occurrence. The number of rows removed is logged as a data quality warning. The demo run found and removed 9 duplicate IDs.

- Step 3 - Missing numeric fill: session_attended_min, practice_questions, and attendance_days columns are filled with 0 for students who had no matching metrics rows (left join artefact). Students with zero activity receive the worst possible component scores, which is the conservative safe default.

- Step 4 - Derived column computation: attendance_rate (fraction of expected sessions attended), avg_practice_questions (mean practice Q per day), trend_direction (improving / stable / declining based on recent vs earlier session data), and days_since_last_note are computed and appended as new columns before the DataFrame is returned.

Missing Data Strategy

Missing data is handled conservatively: any gap in engagement data is treated as a risk signal

rather than ignored. The strategy is deliberately simple - fill with zero or worst-case value - to avoid false negatives where a student appears low-risk only because their data was not recorded.

- Missing session_attended_min - filled with 0, contributing maximum attendance component score (highest risk) rather than being excluded from scoring.

- Missing practice_questions - filled with 0, same logic as session fill.

- Missing facilitator notes (no note rows) - days_since_last_note is set to a large sentinel value (e.g., 999) so the notes component scores at maximum risk. A student with no notes is assumed to have had no facilitator contact.

- Missing student_name or facilitator_email - row is retained; the empty string propagates to outputs and is visible to the facilitator as a data quality signal.

Edge Cases and Quality Issues

The synthetic dataset used for development and testing injects a representative set of data quality issues to verify that the pipeline handles them without crashing. All injected anomalies are logged at WARNING level; the pipeline continues in all cases.

- Duplicate student IDs - 9 rows injected in the demo dataset; all deduplicated on student_id, keeping first occurrence. Logged as: "Removed N duplicate student IDs".

- Type-mismatch strings in numeric columns - approximately 84 cells set to strings like "none" or "abc" in session_attended_min and practice_questions. Converted to NaN via pd.to_numeric(errors='coerce'), then filled with 0.

- Blank numeric cells - approximately 210 cells blanked across metric columns (5% of 4,200 cells). Treated identically to type-mismatch strings after coercion.

- Students with no metrics rows - valid student_id in metadata but no matching rows in daily_metrics.csv. Produced by the left join; session and practice columns filled with 0 post-merge.

- Students with no notes rows - valid student_id in metadata but no rows in facilitator_notes.csv. days_since_last_note set to sentinel value for maximum risk contribution.

Merge Logic

The three CSVs are merged in two steps using student_id as the join key. The metadata CSV is the left table in both joins, ensuring every student appears in the output even if they have no metrics or notes data.

- Merge 1: student_metadata LEFT JOIN aggregated daily_metrics on student_id. daily_metrics rows are aggregated by student_id before the join (sum for totals, mean for averages) so the join is always 1:1.

- Merge 2: result of Merge 1 LEFT JOIN aggregated facilitator_notes on student_id. Notes are aggregated to a single most-recent note date per student before joining.

- Post-merge validation: assert no student_id duplicates exist in the final DataFrame; log total row count; log count of students missing metrics data; log count of students missing notes data.